

Cómo funciona el algoritmo FOIL

Análisis del código Python: inducción de reglas y cálculo de la ganancia FOIL

1. Qué es FOIL

FOIL (*First Order Inductive Learner*, Quinlan, 1990) es un algoritmo de aprendizaje inductivo de reglas. Dado un conjunto de ejemplos **positivos** (los que cumplen el concepto a aprender) y **negativos** (los que no), busca una regla del tipo "si se cumplen las condiciones C1, C2, ... entonces el ejemplo es positivo".

Su idea central es un ciclo **voraz (greedy)** de *cobertura secuencial*:

- Se parte de una regla vacía, que cubre todos los ejemplos (positivos y negativos).
- Se agrega, de a una, la condición que más mejora la regla, medida con la **ganancia FOIL**.
- Se repite hasta que la regla deja de cubrir negativos.
- Se eliminan los positivos ya cubiertos y se aprende una nueva regla con los restantes.

El código analizado **no implementa este ciclo completo**: contiene una versión simplificada para inducir reglas y, por separado, un cálculo aislado de la ganancia FOIL. En las secciones siguientes se explica cada parte y al final se marcan las diferencias.

2. Estructura del código

El script tiene tres bloques que repiten la misma lógica sobre dos conjuntos de datos distintos:

Bloque	Dataset	Qué hace
1	Personas y <i>en_formacion</i> (atributos: edad, departamento, nivel_educativo)	Induce una regla, lista atributos exclusivos de los positivos y calcula la ganancia de <i>nivel_educativo == 'terciario'</i> .
2	Personas y <i>alumno</i> (atributos: edad, ocupacion, educacion)	Calcula la ganancia de <i>ocupacion == 'estudiante'</i> .
3	Mismo dataset del bloque 2	Vuelve a definir <i>inducir_regla</i> con otros atributos y la ejecuta.

3. Las tres piezas del algoritmo

3.1 Separación de ejemplos

Los datos son una lista de diccionarios. Cada ejemplo tiene sus atributos y una etiqueta booleana (*en_formacion* o *alumno*). Se separan en dos listas:

```
positivos = [p for p in datos if p["en_formacion"]]
negativos = [p for p in datos if not p["en_formacion"]]
```

3.2 *inducir_regla*(positivos, negativos)

Recorre cada atributo y calcula el conjunto de valores que toma en los positivos y en los negativos. Se queda solo con los valores que aparecen **en los positivos pero nunca en los negativos** (diferencia de conjuntos). Si queda al menos un valor, el atributo pasa a formar parte de la regla.

```
valores_pos = set(p[atributo] for p in positivos)
```

```
valores_neg = set(p[atributo] for p in negativos)
valores_validos = list(valores_pos - valores_neg)
if valores_validos:
    regla[atributo] = valores_validos
```

El resultado es un diccionario $\{atributo: [valores\ discriminantes]\}$ que se interpreta como una conjunción: para ser positivo, cada atributo de la regla debe tomar uno de sus valores. Para *edad* el código usa una comprensión de listas en lugar de la resta de conjuntos, pero el efecto es idéntico.

3.3 atributos_exclusivos(datos)

Es una función clasificadora auxiliar que hace lo mismo que *inducir_regla*, pero con los tres atributos escritos a mano y devolviendo tres conjuntos: edades, departamentos y niveles educativos exclusivos de los positivos. Se imprime como información complementaria.

3.4 Ganancia FOIL

Es la métrica que FOIL usa para decidir que condición agregar. Compara la proporción de positivos **antes** y **después** de aplicar la condición, y la pondera por cuántos positivos sobreviven:

$$\text{FOIL Gain} = p \times (\log_2(p / (p + n)) - \log_2(P / (P + N)))$$

Símbolo	Significado
P, N	Positivos y negativos cubiertos antes de agregar la condición
p, n	Positivos y negativos que siguen cubiertos después de la condición
$p / (p + n)$	Precisión de la regla tras la condición
$P / (P + N)$	Precisión de partida

Una ganancia alta significa que la condición aumenta mucho la pureza de la regla y además conserva muchos positivos. La función *log2_safe* devuelve *-inf* si el argumento es 0 para evitar un error de dominio en el logaritmo.

4. Ejemplo 1: personas en formación

#	edad	departamento	nivel_educativo	en_formacion
1	22	IT	terciario	Sí
2	24	IT	universitario	Sí
3	21	RRHH	terciario	Sí
4	35	IT	universitario	No
5	40	Finanzas	maestría	No
6	29	RRHH	universitario	No
7	23	IT	terciario	Sí
8	38	Finanzas	universitario	No

Regla inducida. Aplicando la lógica de la sección 3.2 a cada atributo:

Atributo	Valores en positivos	Valores en negativos	Positivos - negativos
edad	21, 22, 23, 24	29, 35, 38, 40	21, 22, 23, 24

Atributo	Valores en positivos	Valores en negativos	Positivos - negativos
departamento	IT, RRHH	IT, RRHH, Finanzas	(vacío) - no entra
nivel_educativo	terciario, universitario	universitario, maestría	terciario

Regla final: **edad in {21, 22, 23, 24}** y **nivel_educativo = terciario**. El departamento se descarta porque IT y RRHH también aparecen entre los negativos.

Ganancia de la condición nivel_educativo == 'terciario'. Antes: P = 4, N = 4. Los ejemplos terciarios son los #1, #3 y #7, todos positivos, así que p = 3 y n = 0.

Cálculo	Valor
$p / (p + n)$	$3 / 3 = 1.000$
$P / (P + N)$	$4 / 8 = 0.500$
$\log_2(1.000)$	0.000
$\log_2(0.500)$	-1.000
FOIL Gain	$3 \times (0 - (-1)) = 3.000$

La condición es "perfecta": elimina todos los negativos sin perder ningún positivo que cubra. Esa es la ganancia máxima posible para p = 3 (cada positivo aporta 1 bit de información).

5. Ejemplo 2: alumnos

En el segundo dataset hay 4 positivos y 4 negativos (P = 4, N = 4). La condición evaluada es *ocupacion* == 'estudiante'. Hay 5 estudiantes: 4 son alumnos y 1 no lo es (el ejemplo de 23 años con educacion universitaria), por lo que p = 4 y n = 1.

Cálculo	Valor
$p / (p + n)$	$4 / 5 = 0.800$
$P / (P + N)$	$4 / 8 = 0.500$
$\log_2(0.800)$	-0.322
$\log_2(0.500)$	-1.000
FOIL Gain	$4 \times (-0.322 + 1) = 2.712$

La ganancia (2.712) es menor que en el ejemplo 1 (3.000) porque la condición conserva los 4 positivos pero deja pasar un negativo: la regla no queda totalmente pura.

Regla del bloque 3. Con los atributos edad, ocupacion y educacion:

- **edad:** 19, 20, 21, 22 (ninguna coincide con las edades de los negativos).
- **ocupacion:** "estudiante" también aparece entre los negativos (el ejemplo de 23 años), así que la diferencia de conjuntos es vacía y el atributo se descarta.
- **educacion:** secundaria y terciario no aparecen en los negativos, por lo que ambos entran.

Es un resultado interesante: *ocupacion* == 'estudiante' tiene una buena ganancia FOIL (2.712), pero *inducir_regla* la descarta por completo, porque solo acepta valores 100% exclusivos. Esto muestra que las dos partes del código no están conectadas entre sí.

6. Diferencias con el FOIL real y limitaciones

El código ilustra ideas de FOIL, pero conviene tener presente que es una simplificación:

Aspecto	FOIL original	Este código
Selección de condiciones	Prueba todas las candidatas y elige la de mayor ganancia FOIL en cada paso.	No usa la ganancia para elegir. Incluye todo valor exclusivo de los positivos.
Ganancia	Se recalcula tras cada condición agregada.	Se calcula una sola vez, para una condición elegida a mano, solo para mostrarla.
Cobertura secuencial	Aprende varias reglas, quitando los positivos ya cubiertos.	Produce una única regla; nunca elimina ejemplos.
Condiciones	Literales de lógica de primer orden con variables y relaciones.	Igualdad sobre atributos de una sola tabla (lógica proposicional).
Atributos numéricos	Usa umbrales (por ejemplo, edad < 25).	Trata cada edad como un valor discreto, lo que memoriza los datos en vez de generalizar.

Otros puntos a considerar:

- **Sobreajuste en edad:** la regla enumera las edades exactas vistas (21 a 24). Una persona de 25 años no sería cubierta, aunque probablemente debería. Un umbral sería más robusto.
- **Criterio demasiado estricto:** un valor con 99% de positivos y un solo negativo se descarta, como pasa con *ocupacion* = *estudiante*. No hay tolerancia al ruido.
- **Divisiones por cero:** si $p + n = 0$, la expresión $p / (p + n)$ falla. Si $p = 0$, *log2_safe* devuelve -inf y el producto $0 \times (-inf)$ da *nan*.
- **Código duplicado:** *import math*, *log2_safe*, *datos* e *inducir_regla* se definen más de una vez. Conviene unificarlos en funciones parametrizadas.
- **Orden no determinista:** los valores de texto salen de conjuntos, y su orden puede cambiar entre ejecuciones.

7. Conclusión

El programa muestra dos conceptos fundamentales de FOIL: (1) buscar condiciones que separen positivos de negativos y (2) medir la utilidad de una condición con la ganancia FOIL, que premia aumentar la proporción de positivos sin perder cobertura. En los ejemplos, *nivel_educativo* == 'terciario' obtiene la ganancia máxima (3.000) y *ocupacion* == 'estudiante' una ganancia alta pero menor (2.712).

Para acercarlo al algoritmo real, el siguiente paso sería construir un bucle que, en cada iteración, calcule la ganancia de todas las condiciones candidatas, agregue la mejor a la regla, filtre los ejemplos cubiertos y se detenga cuando no queden negativos; luego repetir con los positivos restantes.